

Chapter 56 — Concurrency Basics for Coding Interviews

0. What You Will Learn in This Chapter

Concurrency problems ask you to coordinate multiple threads safely.

They appear less often than arrays, graphs, or DP, but when they appear, they can be intimidating because normal sequential reasoning is not enough.

Concurrency problems involve:

- multiple threads running at the same time
- shared state
- race conditions
- ordering constraints
- locks
- condition variables
- semaphores
- blocking queues
- deadlock avoidance
- atomic operations

By the end of this chapter, you should understand:

- What concurrency is
- What a race condition is
- Why shared state is dangerous
- How mutexes work
- How condition variables work
- How semaphores work
- How to coordinate thread ordering
- How to solve Print in Order
- How to solve FooBar Alternately
- How to solve Zero Even Odd
- How to design a blocking queue
- How to reason about Dining Philosophers
- What deadlock is
- How to avoid deadlock
- When atomic variables are useful
- Common concurrency interview traps

Core problems covered in this chapter:

1. Print in Order
2. Print FooBar Alternately
3. Print Zero Even Odd
4. Building H2O
5. Fizz Buzz Multithreaded
6. Design Bounded Blocking Queue
7. Dining Philosophers
8. Traffic Light Controlled Intersection
9. Web Crawler Multithreaded, preview
10. Producer Consumer Pattern
11. Reader Writer Lock, preview
12. Mutex basics

- 13. Condition variable basics
 - 14. Semaphore basics
 - 15. Atomic counter basics
-

1. What Is Concurrency?

Concurrency means multiple tasks can make progress during overlapping time periods.

In programming, this often means multiple threads execute code that may interact with shared data.

Example:

Thread A increments counter.
Thread B increments counter.

If both access the same counter without synchronization, the final value may be wrong.

Concurrency is difficult because thread execution order is not deterministic.

The same code can produce different results across runs.

2. Thread vs Process

2.1 Process

A process is an independent running program.

Processes usually have separate memory spaces.

Example:

Chrome process
VS Code process
terminal process

2.2 Thread

A thread is a unit of execution inside a process.

Threads in the same process share memory.

This shared memory is powerful but dangerous.

If two threads modify the same data without coordination, bugs can happen.

3. Shared State

Shared state is data accessed by multiple threads.

Examples:

```
int counter;  
queue<int> q;  
vector<int> data;  
unordered_map<int, int> mp;
```

If one thread writes while another reads or writes, you need synchronization.

Shared state is the root of most concurrency bugs.

4. Race Condition

A race condition happens when program correctness depends on unpredictable thread timing.

Example:

```
counter++;
```

This looks like one operation, but it is actually multiple steps:

```
read counter  
add 1  
write counter
```

If two threads do this at the same time, both may read the same old value and overwrite each other.

Example:

counter starts at 0

Thread A reads 0

Thread B reads 0

Thread A writes 1

Thread B writes 1

Expected:

2

Actual:

1

That is a race condition.

5. Critical Section

A critical section is code that accesses shared state and must not be executed by multiple threads at the same time.

Example:

```
lock mutex  
counter++  
unlock mutex
```

Only one thread can execute the protected section at once.

6. Mutex

A mutex is a mutual exclusion lock.

It allows only one thread to enter a critical section.

C++:

```
mutex mtx;

void increment() {
    lock_guard<mutex> lock(mtx);
    counter++;
}
```

lock_guard automatically locks when created and unlocks when it goes out of scope. This is safer than manually calling:

```
mtx.lock();
mtx.unlock();
```

because exceptions or early returns can skip unlock.

7. unique_lock vs lock_guard

7.1 lock_guard

Simple automatic lock.

```
lock_guard<mutex> lock(mtx);
```

Use when:

- lock is acquired immediately
- lock is released at scope exit
- no need to unlock manually
- no condition variable needed

7.2 unique_lock

More flexible.

```
unique_lock<mutex> lock(mtx);
```

Use when:

- condition variable is needed
- you need to unlock and relock manually
- ownership may be transferred

Condition variables require unique_lock.

8. Condition Variable

A condition variable lets one thread wait until another thread signals that a condition changed.

Common pattern:

```
mutex mtx;
condition_variable cv;
bool ready = false;

void worker() {
    unique_lock<mutex> lock(mtx);
```

```
cv.wait(lock, [] {  
    return ready;  
});  
  
// proceed  
}
```

Another thread:

```
{  
    lock_guard<mutex> lock(mtx);  
    ready = true;  
}  
  
cv.notify_one();
```

The waiting thread sleeps until notified and the condition is true.

9. Why Condition Variables Need a Predicate

Wrong:

```
cv.wait(lock);
```

Better:

```
cv.wait(lock, [] {  
    return condition;  
});
```

Why?

Because of spurious wakeups.

A thread can wake up even if nobody correctly signaled it.

The predicate ensures the thread continues only when the condition is actually true.

This is one of the most important concurrency interview details.

10. notify_one vs notify_all

10.1 notify_one

Wakes one waiting thread.

Use when only one thread can proceed.

Example:

```
one item added to queue
```

10.2 notify_all

Wakes all waiting threads.

Use when multiple threads may now be able to proceed, or when you are unsure which type of waiter needs to wake.

Example:

```
state changed globally
shutdown requested
```

11. Semaphore

A semaphore controls access to a limited number of permits.

Conceptually:

```
acquire permit before proceeding
release permit after done
```

If no permit is available, thread waits.

A binary semaphore has capacity 1 and acts somewhat like a mutex.

A counting semaphore can allow multiple threads.

C++20 has:

```
counting_semaphore
binary_semaphore
```

LeetCode concurrency problems often provide a semaphore-like abstraction in some languages.

12. Atomic Variables

Atomic variables support thread-safe operations without explicit locks.

Example:

```
atomic<int> counter = 0;

counter++;
```

This increment is atomic.

Use atomics for simple shared values such as:

- counters
- flags
- sequence numbers

Do not use atomics as a replacement for mutexes when multiple related variables must be updated together.

Example needing mutex:

```
if (!q.empty()) {
    int x = q.front();
    q.pop();
}
```

The check and pop must be protected together.

13. Deadlock

Deadlock happens when threads wait forever for each other.

Example:

Thread A holds lock1 and waits for lock2.

Thread B holds lock2 and waits for lock1.

Neither can proceed.

Program freezes.

14. Conditions for Deadlock

Classic deadlock conditions:

16. Mutual exclusion
17. Hold and wait
18. No preemption
19. Circular wait

To avoid deadlock, break at least one condition.

Most common technique:

Always acquire locks in the same global order.

15. Common Synchronization Tools

mutex	protect shared state
lock_guard	simple scoped lock
unique_lock	condition variable wait
condition_variable	wait until condition is true
semaphore	permit-based ordering or capacity
atomic	simple lock-free counters/flags
queue + cv	producer-consumer
ordered locks	deadlock prevention

16. Pattern Recognition Checklist

Ask:

20. Are multiple threads calling different methods?
21. Is there shared state?
22. Is a specific execution order required?
23. Do threads need to alternate?

24. Is there a bounded capacity?
 25. Can producers wait when full?
 26. Can consumers wait when empty?
 27. Is there risk of deadlock?
 28. Can a semaphore model the order?
 29. Do I need condition variables?
 30. Is atomic enough, or do multiple variables need consistency?
-

17. Pattern 1 — Ordering Threads

Use this when methods must execute in a fixed order.

Example:

```
first()
second()
third()
```

Possible tools:

- condition variable with state
- semaphores
- promise/future
- atomic stage variable with waiting

Simple condition variable pattern:

```
int stage = 1;
mutex mtx;
condition_variable cv;
```

Each method waits for its stage, runs, increments stage, notifies.

18. Full Problem Walkthrough 1 — LeetCode 1114: Print in Order

18.1 Problem Summary

Three methods may be called by different threads:

```
first()
second()
third()
```

Need guarantee output:

```
firstsecondthird
```

18.2 Condition Variable Approach

State:

```
stage = 1
```

- first waits for stage 1, prints, sets stage 2.
- second waits for stage 2, prints, sets stage 3.

- third waits for stage 3, prints.

18.3 C++ Code

```
class Foo {
private:
    mutex mtx;
    condition_variable cv;
    int stage;

public:
    Foo() {
        stage = 1;
    }

    void first(function<void()> printFirst) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return stage == 1;
        });

        printFirst();

        stage = 2;

        cv.notify_all();
    }

    void second(function<void()> printSecond) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return stage == 2;
        });

        printSecond();

        stage = 3;

        cv.notify_all();
    }

    void third(function<void()> printThird) {
```

```
unique_lock<mutex> lock(mtx);

cv.wait(lock, [&] {
    return stage == 3;
});

printThird();
}
};
```

18.4 Complexity

Each method:

$O(1)$

19. Pattern 2 — Alternating Threads

Use when two threads must alternate.

Example:

foo bar foo bar foo bar

State:

turn = 0 for foo
turn = 1 for bar

Each method waits until it is its turn, prints, flips turn, notifies.

20. Full Problem Walkthrough 2 — LeetCode 1115: Print FooBar Alternately

20.1 Problem Summary

Two methods:

foo()
bar()

Need print:

foobarfoobar...

n times.

20.2 C++ Code

```
class FooBar {
private:
```

```
int n;  
mutex mtx;  
condition_variable cv;  
bool fooTurn;
```

```
public:
```

```
    FooBar(int n) {  
        this->n = n;  
        fooTurn = true;  
    }
```

```
    void foo(function<void()> printFoo) {  
        for (int i = 0; i < n; i++) {  
            unique_lock<mutex> lock(mtx);  
  
            cv.wait(lock, [&] {  
                return fooTurn;  
            });  
  
            printFoo();  
  
            fooTurn = false;  
  
            cv.notify_all();  
        }  
    }
```

```
    void bar(function<void()> printBar) {  
        for (int i = 0; i < n; i++) {  
            unique_lock<mutex> lock(mtx);  
  
            cv.wait(lock, [&] {  
                return !fooTurn;  
            });  
  
            printBar();  
  
            fooTurn = true;  
  
            cv.notify_all();  
        }  
    }
```

```
};
```

20.3 Semaphore Alternative

If semaphores are available:

fooSem starts with 1 permit
barSem starts with 0 permits

Foo acquires fooSem, prints, releases barSem.
Bar acquires barSem, prints, releases fooSem.

21. Full Problem Walkthrough 3 — LeetCode 1116: Print Zero Even Odd

21.1 Problem Summary

There are three methods:

```
zero()  
even()  
odd()
```

For n = 5, output:

```
0102030405
```

Zero prints before every number.
Odd/even number threads print the corresponding number.

21.2 State

Need track:

```
current number  
turn
```

Turns:

```
0 = zero  
1 = odd  
2 = even
```

After zero prints:

- if current number is odd, turn = odd
- if current number is even, turn = even

After number prints:

- increment current
- turn = zero

21.3 C++ Code

```
class ZeroEvenOdd {  
private:
```

```
int n;  
int current;  
int turn;  
  
mutex mtx;  
condition_variable cv;
```

```
public:
```

```
ZeroEvenOdd(int n) {  
    this->n = n;  
    current = 1;  
    turn = 0;  
}
```

```
void zero(function<void(int)> printNumber) {  
    for (int i = 1; i <= n; i++) {  
        unique_lock<mutex> lock(mtx);  
  
        cv.wait(lock, [&] {  
            return turn == 0;  
        });  
  
        printNumber(0);  
  
        if (current % 2 == 1) {  
            turn = 1;  
        } else {  
            turn = 2;  
        }  
  
        cv.notify_all();  
    }  
}
```

```
void odd(function<void(int)> printNumber) {  
    for (int i = 1; i <= n; i += 2) {  
        unique_lock<mutex> lock(mtx);  
  
        cv.wait(lock, [&] {  
            return turn == 1;  
        });  
    }  
}
```

```

        printNumber(current);
        current++;

        turn = 0;

        cv.notify_all();
    }
}

void even(function<void(int)> printNumber) {
    for (int i = 2; i <= n; i += 2) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return turn == 2;
        });

        printNumber(current);
        current++;

        turn = 0;

        cv.notify_all();
    }
}
};

```

22. Pattern 3 — Multiple Conditions

Some problems have several types of worker threads.

Example:

hydrogen and oxygen threads build water

Need allow exactly:

2 hydrogen
1 oxygen

per molecule.

Useful tools:

- semaphores for capacity
- barrier for groups
- counters + condition variables

23. Full Problem Walkthrough 4 — LeetCode 1117: Building H2O

23.1 Problem Summary

Hydrogen thread calls:

```
hydrogen()
```

Oxygen thread calls:

```
oxygen()
```

Need output groups that form water:

```
two H and one O
```

Order inside each group does not matter.

23.2 Semaphore Idea

Use:

```
hydrogen permits = 2
```

```
oxygen permits = 1
```

But after 2 hydrogens and 1 oxygen complete, reset permits.

In C++, this can be done with counters and condition variable.

23.3 Simple Condition Variable Code

```
class H2O {
private:
    mutex mtx;
    condition_variable cv;

    int hCount;
    int oCount;

public:
    H2O() {
        hCount = 0;
        oCount = 0;
    }

    void hydrogen(function<void()> releaseHydrogen) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return hCount < 2;
        });
```

```

    releaseHydrogen();

    hCount++;

    if (hCount == 2 && oCount == 1) {
        hCount = 0;
        oCount = 0;
    }

    cv.notify_all();
}

void oxygen(function<void()> releaseOxygen) {
    unique_lock<mutex> lock(mtx);

    cv.wait(lock, [&] {
        return oCount < 1;
    });

    releaseOxygen();

    oCount++;

    if (hCount == 2 && oCount == 1) {
        hCount = 0;
        oCount = 0;
    }

    cv.notify_all();
}
};

```

23.4 Important Note

This ensures the counts do not exceed 2 H and 1 O per group. Some stricter versions require a barrier so the next molecule cannot start until all three threads from the current molecule finish. The above is usually enough for basic ordering constraints, but a production-quality barrier is more precise.

24. Pattern 4 — FizzBuzz with Threads

Multiple threads print based on number divisibility.
Need ensure each number is handled exactly once.

State:

current number

Each thread waits until current matches its condition.
After printing, increment current and notify all.

25. Full Problem Walkthrough 5 — LeetCode 1195: Fizz Buzz Multithreaded

25.1 Problem Summary

Four methods:

```
fizz()
buzz()
fizzbuzz()
number()
```

For numbers 1 to n:

- divisible by 3 only: fizz
- divisible by 5 only: buzz
- divisible by both: fizzbuzz
- otherwise: number

25.2 C++ Code

```
class FizzBuzz {
private:
    int n;
    int current;

    mutex mtx;
    condition_variable cv;

public:
    FizzBuzz(int n) {
        this->n = n;
        current = 1;
    }

    void fizz(function<void()> printFizz) {
        while (true) {
            unique_lock<mutex> lock(mtx);

            cv.wait(lock, [&] {
```

```

        return current > n || (current % 3 == 0 && current % 5 != 0);
    });

    if (current > n) {
        cv.notify_all();
        return;
    }

    printFizz();
    current++;

    cv.notify_all();
}

}

void buzz(function<void()> printBuzz) {
    while (true) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return current > n || (current % 5 == 0 && current % 3 != 0);
        });

        if (current > n) {
            cv.notify_all();
            return;
        }

        printBuzz();
        current++;

        cv.notify_all();
    }
}

void fizzbuzz(function<void()> printFizzBuzz) {
    while (true) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return current > n || (current % 15 == 0);
        });
    }
}

```

```

        if (current > n) {
            cv.notify_all();
            return;
        }

        printFizzBuzz();
        current++;

        cv.notify_all();
    }
}

void number(function<void(int)> printNumber) {
    while (true) {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return current > n || (current % 3 != 0 && current % 5 != 0);
        });

        if (current > n) {
            cv.notify_all();
            return;
        }

        printNumber(current);
        current++;

        cv.notify_all();
    }
}
};

```

25.3 Key Detail

Every waiting predicate includes:

```
current > n
```

so threads can exit instead of waiting forever.

26. Pattern 5 — Producer Consumer

Producer-consumer is one of the most important concurrency patterns.

There is a shared queue.

Producers add items.

Consumers remove items.

If queue is bounded:

- producer waits when full
- consumer waits when empty

Need:

mutex

condition_variable notFull

condition_variable notEmpty

27. Full Problem Walkthrough 6 — Design Bounded Blocking Queue

27.1 Problem Summary

Implement bounded blocking queue with:

enqueue(element)

dequeue()

size()

Rules:

- enqueue blocks if queue is full.
- dequeue blocks if queue is empty.

27.2 C++ Code

```
class BoundedBlockingQueue {
private:
    queue<int> q;
    int capacity;

    mutex mtx;
    condition_variable notFull;
    condition_variable notEmpty;

public:
    BoundedBlockingQueue(int capacity) {
        this->capacity = capacity;
    }

    void enqueue(int element) {
        unique_lock<mutex> lock(mtx);
```

```

    notFull.wait(lock, [&] {
        return q.size() < capacity;
    });

    q.push(element);

    notEmpty.notify_one();
}

int dequeue() {
    unique_lock<mutex> lock(mtx);

    notEmpty.wait(lock, [&] {
        return !q.empty();
    });

    int value = q.front();
    q.pop();

    notFull.notify_one();

    return value;
}

int size() {
    lock_guard<mutex> lock(mtx);

    return q.size();
}
};

```

27.3 Why Two Condition Variables?

notFull is for producers.

notEmpty is for consumers.

This avoids waking irrelevant threads more often than necessary.

28. Pattern 6 — Dining Philosophers

Dining Philosophers is a classic deadlock problem.

There are 5 philosophers and 5 forks.

Each philosopher needs left and right fork to eat.

Naive approach:

each philosopher picks left fork, then right fork
can deadlock if everyone picks left fork first.

29. Full Problem Walkthrough 7 — Dining Philosophers

29.1 Problem Summary

Implement:

```
wantsToEat(philosopher, pickLeftFork, pickRightFork, eat, putLeftFork, putRightFork)
```

Need avoid deadlock.

29.2 Deadlock Avoidance Strategy

Always acquire lower-numbered fork first, then higher-numbered fork.

This creates a global lock order and prevents circular wait.

29.3 C++ Code

```
class DiningPhilosophers {
private:
    mutex forks[5];

public:
    DiningPhilosophers() {
    }

    void wantsToEat(int philosopher,
                    function<void()> pickLeftFork,
                    function<void()> pickRightFork,
                    function<void()> eat,
                    function<void()> putLeftFork,
                    function<void()> putRightFork) {
        int left = philosopher;
        int right = (philosopher + 1) % 5;

        int first = min(left, right);
        int second = max(left, right);

        unique_lock<mutex> lock1(forks[first]);
        unique_lock<mutex> lock2(forks[second]);

        if (first == left) {
            pickLeftFork();
```

```

        pickRightFork();
    } else {
        pickRightFork();
        pickLeftFork();
    }

    eat();

    putLeftFork();
    putRightFork();
}
};

```

29.4 Alternative Strategy

Allow at most 4 philosophers to try eating at once using a semaphore.
Then at least one philosopher can always get both forks.

30. Full Problem Walkthrough 8 — Traffic Light Controlled Intersection

30.1 Problem Summary

Two roads share an intersection.
Only one road can have green light at a time.
Cars call:

```
carArrived(carId, roadId, direction, turnGreen, crossCar)
```

Need ensure light turns green for the correct road before car crosses.

30.2 Key Idea

Protect current green road with mutex.
If arriving road is different, call turnGreen.
Then call crossCar.

30.3 C++ Code

```

class TrafficLight {
private:
    int greenRoad;
    mutex mtx;

public:
    TrafficLight() {
        greenRoad = 1;
    }

    void carArrived(int carId,

```

```

        int roadId,
        int direction,
        function<void()> turnGreen,
        function<void()> crossCar) {
    lock_guard<mutex> lock(mtx);

    if (greenRoad != roadId) {
        turnGreen();
        greenRoad = roadId;
    }

    crossCar();
}
};

```

30.4 Lesson

Not every concurrency problem needs condition variables.
Sometimes a mutex is enough.

31. Reader-Writer Lock Preview

A reader-writer lock allows:

- multiple readers at the same time
- only one writer
- writer excludes readers and other writers

Useful when reads are frequent and writes are rare.

Concept:

read lock: shared access
write lock: exclusive access

C++17 has:

shared_mutex
shared_lock
unique_lock

Example:

```

shared_mutex smtx;

void read() {
    shared_lock<shared_mutex> lock(smtx);
    // read shared data
}

void write() {
    unique_lock<shared_mutex> lock(smtx);
}

```

```
// modify shared data
}
```

32. Atomic Counter Example

If all you need is a thread-safe counter:

```
class AtomicCounter {
private:
    atomic<int> count;

public:
    AtomicCounter() {
        count = 0;
    }

    void increment() {
        count++;
    }

    int get() {
        return count.load();
    }
};
```

But if you need to check and update multiple pieces of state together, use a mutex.

33. Common Concurrency Edge Cases

33.1 Waiting Forever

Threads can wait forever if no one notifies or if exit condition is missing.

Example:

In FizzBuzz, include:

```
current > n
```

in every predicate.

33.2 Spurious Wakeups

Always use predicate form:

```
cv.wait(lock, predicate);
```

33.3 Notify After State Change

Correct:

change state
notify

If you notify before changing state, waiting threads may wake and still see old state.

33.4 Holding Lock Too Long

Do not do expensive work while holding lock unless necessary.

For interview problems, printing is often done while holding lock to preserve order.

In production, minimize locked sections.

33.5 Multiple Locks

Always acquire multiple locks in consistent order.

34. Common Concurrency Mistakes

Mistake 1 — Using if Instead of wait Predicate

Wrong:

```
if (!ready) {  
    cv.wait(lock);  
}
```

Correct:

```
cv.wait(lock, [&] {  
    return ready;  
});
```

Mistake 2 — Forgetting notify

If state changes but no thread is notified, other threads may sleep forever.

Mistake 3 — Race on Shared Variable

Reading or writing shared state without lock or atomic can race.

Mistake 4 — Deadlock From Lock Ordering

Acquiring locks in different orders can deadlock.

Mistake 5 — Missing Exit Condition

Threads waiting after work is complete need a way to exit.

Mistake 6 — Assuming Thread Scheduling Order

Never rely on the OS to run threads in a specific order.

35. Interview Explanation

Templates

35.1 Mutex

I use a mutex to protect shared state so only one thread can read or modify it at a time.

35.2 Condition Variable

I use a condition variable so a thread can sleep until the state indicates it is allowed to proceed. I use the predicate form to handle spurious wakeups.

35.3 Alternating Output

I keep a shared turn variable. Each thread waits until it is its turn, prints, updates the turn, and notifies the other waiting threads.

35.4 Blocking Queue

I use a mutex to protect the queue and two condition variables: one for not empty and one for not full. Producers wait when full, consumers wait when empty.

35.5 Deadlock Avoidance

When multiple locks are needed, I acquire them in a consistent global order to avoid circular wait.

36. Complexity Patterns

Print in Order	$O(1)$ per method	$O(1)$
FooBar Alternately	$O(n)$ total	$O(1)$
Zero Even Odd	$O(n)$ total	$O(1)$
FizzBuzz Multithreaded	$O(n)$ total	$O(1)$
Bounded Blocking Queue	$O(1)$ per operation	$O(\text{capacity})$
Dining Philosophers	$O(1)$ per eat attempt	$O(1)$
Traffic Light	$O(1)$ per car	$O(1)$
Atomic Counter	$O(1)$	$O(1)$

Concurrency complexity usually ignores waiting time because waiting depends on scheduling.

37. Expanded Practice Set

37.1 Must-Do Medium

31. Print in Order
32. Print FooBar Alternately
33. Print Zero Even Odd
34. Building H₂O
35. Fizz Buzz Multithreaded
36. Design Bounded Blocking Queue

37.2 Strong Interview Practice

37. Dining Philosophers
38. Traffic Light Controlled Intersection
39. Producer Consumer with shutdown
40. Reader Writer Lock implementation
41. Thread-safe LRU Cache
42. Thread-safe Hit Counter

37.3 Hard / Stretch

- 43. Web Crawler Multithreaded
 - 44. Thread Pool design
 - 45. Future / Promise implementation preview
 - 46. Barrier implementation
 - 47. Semaphore implementation with mutex and condition variable
 - 48. Lock-free queue preview
-

38. Suggested Study Order

Step 1 — Basic Ordering

Solve:

- 49. Print in Order
- 50. FooBar Alternately

Goal:

Understand condition variables and turn state.

Step 2 — Multiple Thread Coordination

Solve:

- 51. Zero Even Odd
- 52. FizzBuzz Multithreaded
- 53. Building H2O

Goal:

Use predicates for multiple conditions.

Step 3 — Producer Consumer

Solve:

- 54. Bounded Blocking Queue

Goal:

Master queue + mutex + condition variables.

Step 4 — Deadlock Avoidance

Solve:

- 55. Dining Philosophers

Goal:

Understand lock ordering.

Step 5 — Practical Thread-Safe Design

Implement:

- 56. thread-safe counter
- 57. thread-safe queue
- 58. thread-safe cache

Goal:

Apply synchronization to data structures.

39. Pattern Decision Table

one method before another	stage variable + cv

Problem Clue	Technique
two methods alternate	turn variable + cv
multiple print conditions	current number + cv
producer waits if full	notFull condition variable
consumer waits if empty	notEmpty condition variable
shared state update	mutex
simple counter	atomic
multiple locks	global lock ordering
limited concurrent access	semaphore
many readers, few writers	shared_mutex
avoid busy waiting	condition_variable

40. Mini Problem 1 — Thread-Safe Counter

```

class Counter {
private:
    int value;
    mutex mtx;

public:
    Counter() {
        value = 0;
    }

    void increment() {
        lock_guard<mutex> lock(mtx);
        value++;
    }

    int get() {
        lock_guard<mutex> lock(mtx);
        return value;
    }
};

```

Atomic version:

```
class AtomicCounter {  
private:  
    atomic<int> value;  
  
public:  
    AtomicCounter() {  
        value = 0;  
    }  
  
    void increment() {  
        value++;  
    }  
  
    int get() {  
        return value.load();  
    }  
};
```

41. Mini Problem 2 — Thread-Safe Queue

```
class ThreadSafeQueue {  
private:  
    queue<int> q;  
    mutex mtx;  
  
public:  
    void push(int x) {  
        lock_guard<mutex> lock(mtx);  
        q.push(x);  
    }  
  
    bool tryPop(int& value) {  
        lock_guard<mutex> lock(mtx);  
  
        if (q.empty()) {  
            return false;  
        }  
    }  
};
```

```
    value = q.front();
    q.pop();

    return true;
}
};
```

Blocking version needs a condition variable.

42. Mini Problem 3 — Simple Semaphore Using Mutex and Condition Variable

```
class Semaphore {
private:
    mutex mtx;
    condition_variable cv;
    int permits;

public:
    Semaphore(int count) {
        permits = count;
    }

    void acquire() {
        unique_lock<mutex> lock(mtx);

        cv.wait(lock, [&] {
            return permits > 0;
        });

        permits--;
    }

    void release() {
        {
            lock_guard<mutex> lock(mtx);
            permits++;
        }

        cv.notify_one();
    }
};
```

```
}  
};
```

43. Mini Problem 4 — Simple Barrier

A barrier waits until n threads arrive, then releases all.

```
class Barrier {  
private:  
    mutex mtx;  
    condition_variable cv;  
    int total;  
    int arrived;  
  
public:  
    Barrier(int n) {  
        total = n;  
        arrived = 0;  
    }  
  
    void wait() {  
        unique_lock<mutex> lock(mtx);  
  
        arrived++;  
  
        if (arrived == total) {  
            cv.notify_all();  
        } else {  
            cv.wait(lock, [&] {  
                return arrived == total;  
            });  
        }  
    }  
};
```

Note:

This simple barrier is one-time use.

Reusable barriers need generation counters.

44. Debugging Concurrency Problems

If concurrency code fails, check:

59. Is every shared variable protected by mutex or atomic?
 60. Are condition variable waits using predicates?
 61. Does every state change notify waiting threads?
 62. Is notification happening after state change?
 63. Can a thread wait forever after work ends?
 64. Are all legal turns represented?
 65. Are multiple locks acquired in a consistent order?
 66. Are you holding a lock while calling code that might call back?
 67. Are queue empty/full checks protected by the same mutex?
 68. Are you using `notify_one` where `notify_all` is safer?
 69. Is the code relying on thread scheduling order?
-

45. What to Master Before Chapter 57

Before moving to Advanced DP Review / Mixed Patterns, make sure you can:

- Explain race conditions
 - Explain critical sections
 - Use mutex and `lock_guard`
 - Use `unique_lock` with `condition_variable`
 - Explain spurious wakeups
 - Use wait predicates correctly
 - Solve Print in Order
 - Solve FooBar Alternately
 - Solve Zero Even Odd
 - Solve FizzBuzz Multithreaded
 - Implement Bounded Blocking Queue
 - Explain producer-consumer
 - Explain deadlock
 - Avoid Dining Philosophers deadlock
 - Know when atomic is enough
 - Know when mutex is required
-

46. Final Cheat Sheet

Mutex

```
lock_guard<mutex> lock(mtx);
```

Condition Variable Wait

```
unique_lock<mutex> lock(mtx);
```

```
cv.wait(lock, [&] {  
    return condition;  
});
```

State Change + Notify

```
state = newState;  
cv.notify_all();
```

Alternating Threads

```
wait for my turn  
print/do work  
change turn  
notify others
```

Blocking Queue

```
enqueue waits while full  
dequeue waits while empty
```

Deadlock Avoidance

Acquire locks in consistent global order.

Atomic Counter

```
atomic<int> counter;  
counter++;
```

Producer Consumer

```
shared queue  
mutex protects queue  
notFull for producers  
notEmpty for consumers
```

47. Core Takeaway

Concurrency is about controlling access and ordering.
The two biggest questions are:

What shared state exists?
Which thread is allowed to proceed now?

Use:

- mutex to protect shared state
- condition variables to wait for state changes
- semaphores to model permits
- atomics for simple counters or flags
- lock ordering to avoid deadlock

Most interview concurrency problems reduce to:

wait until predicate is true
perform action
update state
notify others

Always use condition-variable predicates, never rely on thread scheduling, and always think about how each waiting thread eventually exits.
